VIT-AP UNIVERSITY

Amaravati, Andhra Pradesh, India

COMPILER DESIGN LAB REPORT

Subject Name: Compiler Design Lab

Course Code: CSE3024

Machine Learning-Based Loop Optimization for High-Performance Compiler Design

Submitted by

Poojith **Regd. No.:** 24BCB7244

Student 2 Name **Regd. No.:** 24BCB7161

Under the Guidance of

Dr. Prakash Kumar Sarangi

School of Computer Science and Engineering (SCOPE)

VIT-AP University

Amaravati, Andhra Pradesh, India

Academic Year 2026–2027

Abstract

High-performance computing programs spend most of their time in loop nests, and the speed of the generated code depends on optimisation choices such as vectorisation, unrolling and the use of the target instruction set. Production compilers make these choices with fixed, hand-written heuristics that cannot adapt to the individual loop. This report designs and evaluates an ML-based loop-optimisation decision model that sits in front of an unmodified GCC compiler.

A compiler front end is implemented from scratch: a lexical analyser, a recursive-descent parser, a semantic and loop analyser, and a three-address-code generator. From the analysed loop nest the front end computes 23 static features, such as trip counts, exact memory strides, loop-carried dependence distance, reductions, arithmetic intensity and working-set size. A supervised model maps the features to one of 7 GCC optimisation configurations. The model is trained on 182 C loop kernels from 9 HPC-style families (vector updates, reductions, recurrences, 1-D/2-D stencils, matrix multiplication, 2-D traversals, branchy and strided loops). Every kernel was compiled with every configuration (1274 compilations) and executed on the target machine, and the label is the best configuration that preserves the numerical result.

The evaluation uses the speed-up of the generated code. With random 5-fold cross-validation the best model (k-NN) reaches a geometric-mean speed-up of 2.221× over `-O2`, which is 95.0% of the gain of the oracle (2.316×). This must be compared honestly with static choices: always using `-O3` gives 1.054×, always using `-O3 -march=native` gives 1.512×, and always using `-O3 -march=native -ffast-math` gives 2.197×. On this machine the learned model is therefore only marginally better than the most aggressive static configuration in cross-validation, and on loop families never seen in training (leave-one-family-out) it is worse (1.794×), which shows the limits of a small corpus. The model is more useful when the binary must be portable and IEEE-safe: among the four non-native, non-fast-math configurations it reaches 1.108× (oracle 1.175×) against 1.054× for `-O3`, which causes 32 slow-downs of more than 5%. Feature extraction and prediction take about 0.7 ms and 1.3 ms per loop, small compared with a GCC compilation (71 ms).

Keywords: Compiler Design, Loop Optimization, Lexical Analysis, Syntax Analysis, Dependence Analysis, Machine Learning, High-Performance Computing, GCC.

1. Introduction

A compiler is a system software component that translates a program written in a source language into an equivalent program in a target language. For high-performance computing (HPC) workloads such as dense linear algebra, stencil codes, signal processing and scientific simulation, almost all of the execution time is spent inside a small number of *loop nests*. The quality of the machine code generated for these loops therefore determines the performance of the whole application [**?**, **?**].

Modern optimising compilers contain many loop transformations: loop unrolling, loop interchange, loop fusion and fission, tiling, loop-invariant code motion, strength reduction and automatic vectorisation [**?**, **?**, **?**]. Whether a transformation pays off depends on the loop's trip count, memory access pattern, data type, data-dependence structure, working-set size and on the target micro-architecture. Production compilers decide with hand-written heuristics and fixed cost models that were tuned once, on a set of benchmarks, for an average machine. These heuristics are cheap but can be far from the best choice for an individual loop, and tuning them by hand is slow and has to be repeated for every new processor [**?**, **?**].

Machine learning offers an alternative. Instead of writing the heuristic, we *learn* it: a number of loops are compiled and timed under different optimisation choices, and a model is trained to predict the best choice from static features of the loop that the compiler front end can already compute [**?**, **?**, **?**]. The learned decision model replaces the hand-written profitability test of the optimiser.

This report presents a complete, working implementation of this idea for the Compiler Design Lab. A small compiler front end (lexical analyser, recursive descent parser, semantic and loop analyser, three-address-code generator) is written from scratch and used to extract 23 numeric features from C loop nests. A supervised model then selects one of 7 GCC optimisation configurations for the loop. The model is trained and evaluated on 182 loop kernels that were actually compiled and executed on the target machine.

1.1 Motivation

Compiler design is usually taught as a fixed pipeline of phases. The motivation of this project is to show that the *outputs* of the early phases (tokens, syntax tree, symbol table, intermediate code) are exactly the information that an optimisation decision needs, and that the last phase, optimisation, can be driven by a data-driven model instead of an ad-hoc rule. A practical understanding of this connection links compiler theory with the current research direction of ML-guided compilation used in industrial toolchains [**?**, **?**].

The topic is also relevant because HPC users routinely tune compiler flags by trial and error: compile with `-O3`, then with `-march=native`, then with unrolling, and measure. A model that predicts a good configuration directly from the source of a loop can save this search time.

1.2 Problem Statement

Given the source code of a loop nest, the compiler must decide which optimisation configuration to apply. This report addresses the following problem:

> *Given a C loop nest L, can a decision model that uses only static features computed by a compiler front end predict which of K candidate optimisation configurations gives the lowest execution time, so that the compiled code is faster than with the default configuration (`-O2`) and close to the best possible (oracle) configuration?*

Formally, let $C = \{c_1, \ldots, c_K\}$ be the configurations and $T(L, c)$ the measured execution time of loop L compiled with c. We want a function $f : \mathbb{R}^d \to C$ on the feature vector $\phi(L) \in \mathbb{R}^d$ with

$$f(\phi(L)) \approx \arg\min_{c \in C} T(L, c). \tag{1}$$

1.3 Objectives

1. To implement the front-end phases of a compiler (lexical, syntax and semantic analysis, intermediate code generation) for a C loop-nest subset.
2. To perform loop analysis (trip counts, memory strides, loop-carried dependences, reductions) and convert the result into a numeric feature vector.
3. To build a data set of loop kernels labelled with the best GCC optimisation configuration, obtained by real compilation and timing.
4. To train and compare ML decision models, and to evaluate them by the speed-up of the generated code and not only by classification accuracy.
5. To package the result as a command-line tool that advises the compiler flags for a new loop.

1.4 Contributions

The major contributions of this report are as follows:

- **Compiler front end for loop analysis:** A lexer, recursive-descent parser, semantic analyser and TAC generator for a C loop subset, implemented from scratch in Python (Section **??**).
- **Loop feature set:** 23 static features derived from the AST and TAC, including exact memory strides computed from array declarations and dependence distances computed from affine subscripts.

- **Measured data set:** 182 kernels in 9 families, each compiled with 7 configurations and timed with an interleaved, noise-aware protocol (1274 compilations).
- **Performance evaluation of the decision model:** Evaluation by realised speed-up with two protocols, random cross-validation and leave-one-family-out, against static and oracle baselines (Section **??**).

2. Related Work

Compiler design has been extensively studied since the development of the first automatic programming systems. Aho et al. presented comprehensive foundations for lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and code generation [**?**]. Cooper and Torczon described optimization techniques and their implementation in modern compiler infrastructures [**?**]. Appel discussed compiler construction and code generation techniques [**?**].

2.1 Traditional Compiler Models

Traditional compiler architectures use a sequence of analysis and synthesis phases. Lexical analysis converts character sequences into tokens. Syntax analysis organizes tokens into parse trees. Semantic analysis verifies contextual correctness. Intermediate representations are then optimized before target-code generation. LR and LL parsing techniques have been extensively used for context-free grammars; the front end in this report uses a recursive-descent (LL) parser with precedence climbing for expressions.

2.2 Compiler Optimization and Loop Transformations

Optimization attempts to improve program performance while preserving program semantics. Classical techniques include constant folding, constant propagation, common subexpression elimination, dead-code elimination, loop optimization, strength reduction, and control-flow optimization [**?**]. For loops, the central transformations are *unrolling* (replicating the body to reduce loop overhead and expose instruction-level parallelism), *interchange* and *tiling* (reordering iterations to improve cache locality [**?**]) and *vectorisation* (executing several iterations with one SIMD instruction). Their legality is decided by data-dependence analysis, which computes dependence distances between array references [**?**, **?**]. Polyhedral frameworks automate the search of legal loop transformations [**?**].

Machine-independent optimisation operates on intermediate representations without depending on a particular processor architecture. Machine-dependent optimisation considers target-specific characteristics such as registers, instruction sets (e.g. AVX2/AVX-512), cache behaviour and pipeline constraints.

2.3 Machine Learning in Compiler Optimization

Wang and O'Boyle survey the use of machine learning in compilers and conclude that learning replaces hand-built heuristics for decisions such as unroll factors, inlining, phase ordering and device mapping [**?**]; Ashouri et al. survey compiler auto-tuning [**?**]. Stephenson and Amarasinghe used supervised classification to predict loop unroll factors from loop features [**?**], and Stephenson et al. evolved compiler heuristics with genetic programming [**?**]. Agakov et al. used

learned models to focus iterative optimisation on promising transformations [**?**], and Milepost GCC integrated feature extraction and machine learning into GCC itself [**?**]. Cummins et al. learned optimisation heuristics directly from source text with deep networks [**?**]. MLGO brings learned policies into LLVM [**?**], and TVM uses a learned cost model to tune tensor kernels [**?**].

Most of this work either modifies the compiler internals or needs large benchmark suites. The approach in this report is deliberately lightweight: the decision model sits *in front of* an unmodified compiler (GCC) and chooses among its existing flag sets, which makes it easy to reproduce in a laboratory setting.

2.4 Compiler Technology for Machine Learning

Machine learning frameworks generate computational graphs containing operations such as matrix multiplication, convolution and activation functions. Compiler-based approaches such as XLA and MLIR optimise these graphs before execution [**?**, **?**]. This is the opposite direction of the present work (compilers *for* ML versus ML *in* compilers), but both rely on the same intermediate-representation and loop-optimisation machinery.

2.5 Survey of Related Approaches

Table 1: Survey of Compiler Optimisation Decision Approaches

Approach	Decision	Major Contribution	Limitation
GCC / LLVM `-O` levels	All passes	Mature, fixed heuristics and cost models	Tuned for an average program and machine
Polyhedral (Pluto)	Loop tiling, parallelism	Automatic legal transformation search for affine loops	Restricted to affine loop nests
Unroll-factor prediction [**?**]	Unroll factor	Supervised classification from loop features	Single transformation, internal compiler change
Iterative compilation + ML [**?**]	Transformation sequence	Models focus the search on good sequences	Needs many compile-run cycles
Milepost GCC [**?**]	Flags / passes	Static features and learning inside GCC	Large infrastructure
DeepTune [**?**]	Device mapping, coarsening	Learns features from raw source	Needs large training sets
MLGO [**?**]	Inlining, register allocation	Learned policies in production LLVM	Tied to the compiler version
This work	Flag configuration of loops	Own front end + features + model in front of unmodified GCC	Small corpus, one machine

2.6 Limitations of Existing Approaches

Although learned and heuristic optimisers provide significant performance benefits, several limitations remain:

1. Hand-written heuristics are tuned for an average machine and do not adapt to the loop or to the processor.
2. Machine-specific optimisations (e.g. `-march=native`) reduce portability of the generated binary.
3. Some transformations change floating-point results (e.g. `-ffast-math`), so a decision model must be checked for numerical safety.
4. Learned models can mis-predict on loop patterns that were absent from the training data; this must be tested explicitly.

5. Timing labels are noisy on shared machines, so labelling has to be robust to measurement noise.

6. Optimisation increases compilation effort, and the decision model itself must be cheap compared with compilation.

The proposed methodology addresses items 3–6 explicitly: it rejects configurations that change the numerical result, labels with a noise tolerance, evaluates on unseen loop families, and measures the overhead of the model.

3. Proposed Methodology

This work proposes a compiler-oriented framework in which the analysis phases of a compiler feed a machine-learning model that makes the optimisation decision. The framework has an *offline* part (building the training data) and an *online* part (advising a new loop).

3.1 Proposed Workflow

The overall workflow consists of:

1. Source/Input acquisition (a C function `kernel` with a loop nest).
2. Lexical analysis (tokens).
3. Syntax analysis (abstract syntax tree).
4. Semantic analysis (symbol table, trip counts, strides, dependences).
5. Intermediate representation generation (three-address code).
6. Feature-vector construction.
7. ML decision model $\rightarrow$ optimisation configuration.
8. Target-code generation by GCC with the selected flags.
9. Execution and performance evaluation.

3.2 Proposed Flow Diagram

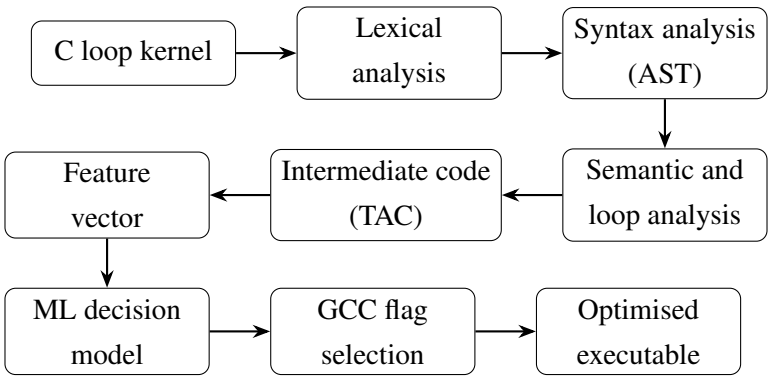


Figure 1: Proposed workflow: compiler front end, feature extraction and ML-based optimisation decision.

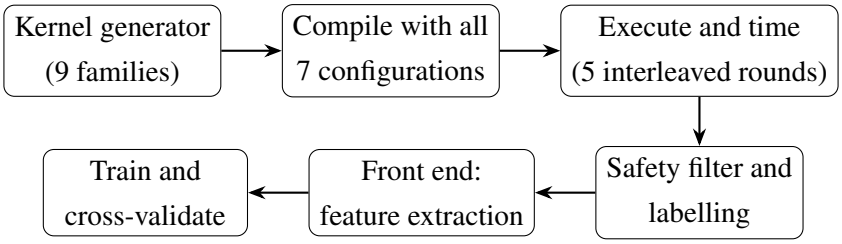


Figure 2: Offline phase: construction of the labelled data set and training.

3.3 Lexical Analysis

Lexical analysis converts a sequence of characters into tokens. The scanner (`lexer.py`) is specified by regular expressions in the same way as a Lex/Flex description, with token classes `KEYWORD`, `TYPE`, `IDENT`, `NUMBER`, `OP`, `PUNCT`; comments, white space and pre-processor lines are discarded and an illegal character is reported as a lexical error. For example, the statement

```
C[i][j] += A[i][k]*B[k][j];
```

is converted into the token sequence (as printed by the implemented scanner):

```
IDENT C | [ i ] [ j ] | OP += | IDENT A | [ i ] [ k ] |
OP * | IDENT B | [ k ] [ j ] | ;
```

3.4 Syntax Analysis

Syntax analysis determines whether the token sequence conforms to the grammar of the language. The output is an Abstract Syntax Tree (AST). The parser (`loop_parser.py`) is a recursive-descent (LL(1)) parser; expressions are parsed by precedence climbing, which corresponds to the usual left-recursion-free expression grammar:

$$E \to T\,((+\,|\,-)\,T)^* \qquad T \to F\,((*\,|\,/)\,F)^*$$

$$F \to \mathit{NUM} \mid \mathit{ID}\,[\,E\,]^* \mid (\,E\,) \mid -F \qquad S \to \texttt{for}\,(\ldots)\,S \mid \texttt{if}\,(E)\,S\,[\texttt{else}\,S] \mid L\ \mathit{op}\ E\,;$$

where $\mathit{op} \in \{=, +=, -=, *=, /=\}$ and L is a scalar or an array element. The AST nodes are `For`, `If`, `Assign`, `BinOp`, `UnOp`, `Index`, `Var`, `Num` and `Call`. Syntax errors are reported with the offending token and line.

3.5 Semantic and Loop Analysis

Semantic analysis checks contextual constraints: every array is declared and used with the declared number of subscripts, every identifier is declared, and loop bounds are constant expressions that can be evaluated from the `#define` constants (a simple symbol table). On the checked AST, the analyser computes the information that drives the optimisation decision.

Trip counts. For `for (i = lo; i < hi; i += s)`, the trip count is $\lceil (hi - lo)/s \rceil$; the iteration space of a nest is the product of the trip counts.

Affine subscripts and strides. Each subscript is evaluated as an affine function $a\,v + b$ of the innermost loop variable v (the function is checked to be linear by evaluating it at three points). For an array with dimensions $S_0, \ldots, S_{n-1}$ and subscripts $e_0, \ldots, e_{n-1}$ with coefficients a_k in v, the memory stride of the reference (in elements) is

$$\text{stride}_v = \sum_{k=0}^{n-1} a_k \prod_{m>k} S_m. \tag{2}$$

A stride of 1 is a unit-stride (vectorisable, cache-friendly) access, a stride of 0 is loop-invariant, and a large stride (e.g. $S_1 = M$ when the *row* index is the inner loop variable) is a cache-hostile access that loop interchange could repair.

Loop-carried dependences. For a write W and a read R of the same array with equal coefficients a and offsets b_W, b_R in the innermost variable, the dependence distance is

$$d = \frac{b_W - b_R}{a}. \tag{3}$$

If $d > 0$ the read uses a value written d iterations earlier (a flow dependence of distance d) and the loop cannot be vectorised with a vector length larger than d.

Reductions. A scalar (or an array element that is invariant in the inner loop) updated with `+=`, `*=` or `x = x + ...` is a reduction. Floating-point reductions can only be vectorised if the compiler may re-associate the additions, which is not allowed under strict IEEE semantics.

3.6 Intermediate Representation

The AST of the loop body is lowered to three-address code (`tac.py`). Each instruction has at most one operator; simple affine subscripts remain inline. Three-address code for the body of the matrix-multiplication loop `C[i][j] += A[i][k]*B[k][j]` is:

$$t_1 = C[i][j] \tag{4}$$
$$t_2 = A[i][k] \tag{5}$$
$$t_3 = B[k][j] \tag{6}$$
$$t_4 = t_2 * t_3 \tag{7}$$
$$t_5 = t_1 + t_4 \tag{8}$$
$$C[i][j] = t_5 \tag{9}$$

The number of TAC instructions of the innermost body is used as a feature (`tac_len`) and the TAC of the complete loop nest, including labels and conditional jumps, can be printed by the tool (Section **??**).

3.7 Feature Vector

The analyser produces the 23 features of Table **??**. They describe the size of the iteration space, the arithmetic and memory work of the body, the access pattern, the dependence structure and the working set.

Table 2: Static loop features used by the decision model.

Feature	**Meaning**
`dtype_bytes`	Size of the data type (4 = float, 8 = double)
`nest_depth`	Depth of the loop nest
`inner_trip_log2`, `total_iters_log2`	$\log_2$ of the innermost trip count and of the total iterations
`n_add`, `n_mul`, `n_div`, `n_flops`	Floating-point operations per innermost iteration (address arithmetic excluded)
`n_loads`, `n_stores`, `n_arrays`	Distinct array loads, stores and arrays per iteration
`n_branches`	Number of `if` statements in the body
`n_unit_stride`, `n_nonunit_stride`, `n_invariant`	Number of references with stride 1, stride $\neq$ 0,1 and stride 0
`max_stride_log2`	$\log_2$ of the largest absolute stride
`has_reduction`	1 if the body contains a reduction
`has_carried_dep`, `dep_distance`	1 if a loop-carried flow dependence exists, and its distance
`footprint_log2`	$\log_2$ of the total size in bytes of the arrays touched
`bytes_per_iter`, `arith_intensity`	Bytes moved per iteration and flops per byte
`tac_len`	Number of TAC instructions in the loop body

3.8 Decision Space

The model chooses one of the 7 GCC configurations of Table **??**. They are ordered from conservative to aggressive: `O2` is the default baseline, the `native` variants use the instruction set of the build machine (less portable), and `O3_fast_native` additionally allows floating-point re-association (`-ffast-math`), which changes numerical results.

Table 3: Decision space: the seven candidate optimisation configurations.

Class	GCC flags
O2	-O2
O3	-O3
O2_unroll4	-O2 -funroll-loops --param max-unroll-times=4
O2_unroll8	-O2 -funroll-loops --param max-unroll-times=8
O3_native	-O3 -march=native
O3_unroll_native	-O3 -funroll-loops -march=native
O3_fast_native	-O3 -march=native -ffast-math

3.9 Labelling Rule

Every kernel is compiled and executed with all configurations. Let T_c be the measured time of configuration c and r_c the checksum of the program output. A configuration is *admissible* if its result agrees with the baseline:

$$\frac{|r_c - r_{O2}|}{|r_{O2}|} \leq 10^{-3}. \tag{10}$$

The label is the most conservative admissible configuration whose time is within $\tau = 5\%$ of the fastest admissible one:

$$y = \min_{\text{order}} \{\, c \;:\; c \text{ admissible},\; T_c \leq (1+\tau) \min_{c' \text{ adm.}} T_{c'} \,\}. \tag{11}$$

The tolerance τ keeps the label stable under measurement noise (Section **??**), and the conservative ordering prefers portable, numerically safe flags whenever a more aggressive flag brings no real gain.

3.10 Decision Models

Four supervised classifiers from scikit-learn [**?**] are compared: a depth-limited *Decision Tree* (interpretable, depth 5), a *Random Forest* [**?**] with 300 trees, *Gradient Boosting* (100 trees of depth 3) and *k-NN* on standardised features ($k = 5$, distance weighting). They are compared with four baselines that do not use the features: always `O2`, always `O3`, always `O3_native` and the majority class.

3.11 Proposed Algorithm

Algorithm 1 ML-Based Loop Optimisation Decision

Require: Loop kernel source S; trained model f; configurations C

Ensure: Compiler flags for S

$tokens \leftarrow \textsc{Lex}(S)$
$ast \leftarrow \textsc{Parse}(tokens)$
Check declarations, subscripts and loop bounds (semantic analysis)
$nest \leftarrow$ loop nest with the largest iteration space
Evaluate trip counts, strides, dependence distance, reductions
$tac \leftarrow \textsc{LowerToTAC}(ast)$
$\phi \leftarrow$ feature vector from $nest$, tac and the array declarations
$c^* \leftarrow f(\phi)$
return $\textsc{Flags}(c^*)$

Algorithm 2 Construction of the Training Set and Model

Require: Kernel corpus $\mathcal{K}$; configurations C; tolerance τ

Ensure: Trained model f

for each kernel $L \in \mathcal{K}$ **do**
    $\phi(L) \leftarrow \textsc{ExtractFeatures}(L)$
    for each configuration $c \in C$ **do**
        compile L with c
    end for
    for round $= 1$ **to** 5 **do**
        for each $c \in C$ **do**
            run the binary of c, record T and the checksum
        end for
    end for
    $T_c \leftarrow$ minimum over the rounds
    $y(L) \leftarrow$ label by the rule above
end for
Train f on $\{(\phi(L), y(L))\}$ and evaluate with cross-validation
return f

3.12 Implementation

The software is organised as follows (all files are provided with this report):

Table 4: Source files of the implementation.

File	Role
`lexer.py`	Phase 1: scanner
`loop_parser.py`	Phase 2: recursive-descent parser, AST
`loop_analyzer.py`	Phase 3: semantic and loop analysis, feature extraction
`tac.py`	Phase 4: three-address-code generator
`kernel_gen.py`	Generator of the kernel corpus and timing harness
`benchmark.py`	Compile/execute under all configurations
`build_dataset.py`	Features, timings, safety filter and labels
`train_model.py`	Training and evaluation of the models
`predict.py`	Command-line advisor for a new loop
`make_report_assets.py`	Generates the tables and plots of this report

4. Results and Discussion

All numbers in this section were produced by the code accompanying this report; the tables and plots are generated automatically from the measured data.

4.1 Dataset Preparation

The corpus consists of 182 C kernels in 9 families (Table **??**). Each kernel is generated by sweeping the data type (`float`/`double`), the problem size, the number of operands, the access stride, the loop order or the dependence distance. Every kernel was compiled with all 7 configurations (1274 compilations) and executed; for each configuration the binary was run in five interleaved rounds (6370 executions in total), and each execution reports the minimum of five timed trials.

Table 5: Composition of the loop-kernel corpus.

Family	Loop pattern	Depth	Kernels	Footprint (bytes)
`axpy`	vector update $y_i = \sum w_j x_i^{(j)} + \ldots$	1	36	2^{13}–2^{24}
`cond`	loop with data-dependent branch	1	16	2^{13}–2^{23}
`dot`	dot product / norm (FP reduction)	1	18	2^{12}–2^{23}
`matmul`	matrix product, loop orders ijk/ikj/jki	3	24	2^{14}–2^{20}
`recur`	first-order recurrence $a_i = f(a_{i-d})$	1	24	2^{13}–2^{23}
`stencil1d`	3- and 5-point 1-D stencil	1	16	2^{13}–2^{23}
`stencil2d`	5-point 2-D Jacobi stencil	2	8	2^{15}–2^{22}
`strided`	non-unit-stride gather $x[s\,i]$	1	20	2^{15}–2^{26}
`traverse2d`	2-D row/column sums, copy, transpose	2	20	2^{17}–2^{24}
Total			**182**	

4.2 System Requirements

Table 6: System used for the experiments.

Component	Value
Processor	Intel(R) Core(TM) i5-6200U CPU @ 2.30GHz (AVX2 and AVX-512 available)
Cores available	4
RAM	3.8 GB
Operating System	Ubuntu 24.04 (Linux)
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
Programming Language	C (kernels), Python 3.12 (front end, ML)
Libraries	NumPy, pandas, scikit-learn

4.3 Evaluation Process

The approach is evaluated with the following metrics. Let $T_c(L)$ be the time of loop L with configuration c, $\hat{c}(L)$ the predicted configuration, and $c^\star(L)$ the fastest admissible configuration (oracle).

4.3.1 Execution Time and Speed-up

$$S(L) = \frac{T_{O2}(L)}{T_{\hat{c}(L)}(L)}, \qquad \mathrm{GM} = \Big(\prod_L S(L)\Big)^{1/N}$$

If the predicted configuration would change the numerical result, the safe baseline `O2` is used instead (S = 1).

4.3.2 Oracle Gain

$$G = \frac{\ln \mathrm{GM}_{\text{model}}}{\ln \mathrm{GM}_{\text{oracle}}} \times 100$$

4.3.3 Regret

$$R = \frac{1}{N} \sum_L \left(\frac{T_{\hat{c}(L)}(L)}{T_{c^\star(L)}(L)} - 1 \right) \times 100$$

4.3.4 Classification Metrics

Accuracy and macro-averaged F1 over the 7 classes are also reported. They are less informative than speed-up because many configurations have almost the same performance for a given loop.

4.3.5 Validation Protocols

Random 5-fold cross-validation (repeated five times) measures interpolation between similar loops. *Leave-one-family-out* (LOFO) trains on eight families and tests on the ninth, which measures generalisation to a new kind of loop and is the stricter test.

4.4 Measurement Noise

The experiments ran on a shared virtual machine, and timings of the *same* binary vary from process to process (probably through physical-page placement and cache conflicts). Across the five rounds, the baseline time of a kernel had a median range of 28.8% (maximum minus minimum). Because the reported time is the minimum over the rounds, the residual noise is smaller: comparing the minimum of the odd rounds with the minimum of the even rounds gives a median difference of 8.6% (90th percentile 22.4%). The 5% tie tolerance of the labelling rule is therefore of the order of the noise, but not above it for all kernels, and some labels, especially

among near-equivalent configurations, are uncertain. This is a limitation of the experiment, discussed in Section **??**.

4.5 Labels and Static Configurations

Table **??** shows how often each configuration is the label and how a *static* choice of that configuration would perform over the whole corpus. No static configuration is best for every kernel (the oracle is above all of them), which is the reason to learn the decision, but the aggressive configurations are close to the oracle on average.

Table 7: Label distribution and performance of each *static* configuration over all kernels (speed-up w.r.t. `O2`; a configuration that changes the numerical result is replaced by `O2`).

Config	Label count	Label %	GM speed-up	Within 5% of best	Rejected
`O2`	25	13.7	1.000	25	0
`O3`	6	3.3	1.054	23	0
`O2_unroll4`	2	1.1	1.084	26	0
`O2_unroll8`	2	1.1	1.109	29	0
`O3_native`	65	35.7	1.512	92	0
`O3_unroll_native`	37	20.3	1.560	114	0
`O3_fast_native`	45	24.7	2.197	128	0
Oracle	–	–	**2.316**	–	–

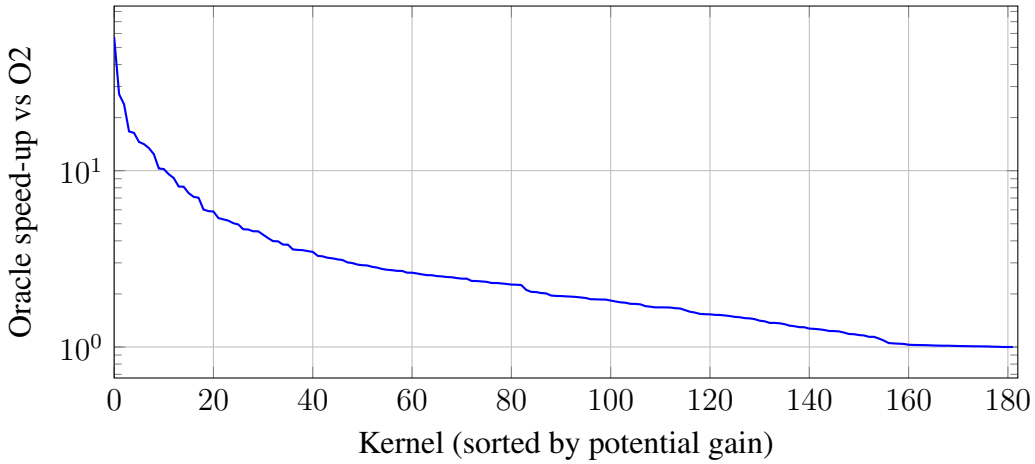


Figure 3: Best achievable speed-up over `O2` for each kernel (oracle).

Figure **??** shows the per-kernel gain that an oracle obtains over `O2`. Many kernels gain from `-march=native`, because the machine supports AVX2 and AVX-512 and the default `x86-64` target does not use them. Reductions (`dot`) gain most from `-ffast-math`, which is consistent with the fact that floating-point reductions can only be vectorised if the additions may be re-associated. The other families show smaller but significant differences between the configurations.

4.6 Performance of the Decision Models

Table 8: Comparison of decision models and static baselines. GM = geometric mean over kernels; "Oracle gain" is the share of the oracle's log-speed-up that is realised; "Slow-downs" counts kernels more than 5% slower than `O2`.

Model	Protocol	Acc. (%)	Macro-F1	GM speed-up	Oracle gain (%)	Regret (%)	Slow-downs
Always -O2	–	13.7	0.03	1.000	0	245.5	0
Always -O3	–	3.3	0.01	1.054	6	232.1	32
Always -O3 -march=native	–	35.7	0.08	1.512	49	170.9	17
Always -O3 -march=native -ffast-math	–	24.7	0.06	2.197	94	6.0	6
Majority class	5-fold CV	35.7 ± 0.0	0.08	1.512	49	170.9	17
Majority class	LOFO	24.7	0.07	1.513	49	170.8	16
Decision Tree	5-fold CV	60.7 ± 2.9	0.34	2.130	90	16.2	6
Decision Tree	LOFO	41.2	0.22	1.706	64	79.8	13
Random Forest	5-fold CV	63.4 ± 1.0	0.38	2.221	95	4.9	6
Random Forest	LOFO	41.2	0.22	1.714	64	78.0	14
Gradient Boosting	5-fold CV	60.2 ± 1.1	0.37	2.175	92	12.9	5
Gradient Boosting	LOFO	32.4	0.19	1.650	60	96.4	15
k-NN	5-fold CV	66.5 ± 1.5	0.41	2.221	95	5.0	6
k-NN	LOFO	48.4	0.27	1.794	70	64.7	12

Table **??** and Figure **??** compare the models with the static baselines. In random cross-validation the learned models use the loop features well: the k-NN reaches 66.5% accuracy and a geometric-mean speed-up of 2.221, i.e. 95.0% of the oracle gain (2.316). Accuracy is only moderate (macro-F1 0.41) because several configurations perform within a few percent of each other; the speed-up is the more relevant quantity.

The comparison with the *best static configuration* is, however, sobering. Always compiling with `-O3 -march=native -ffast-math` already gives 2.197, i.e. 93.7% of the oracle gain, with a regret of 6.0% and 6 slow-downs. The learned model (2.221, regret 5.0%, 6.0 slow-downs) is only marginally better in cross-validation. The reason is that on this processor, which has AVX2 and AVX-512 units, the most aggressive flag set is the right choice for most of the kernels, so there is little left to decide. That static choice is, however, not portable (it uses the instruction set of the build machine) and it relaxes IEEE floating-point semantics; it passed the 10^{-3} checksum test on all kernels here, which does not prove that it is safe in general.

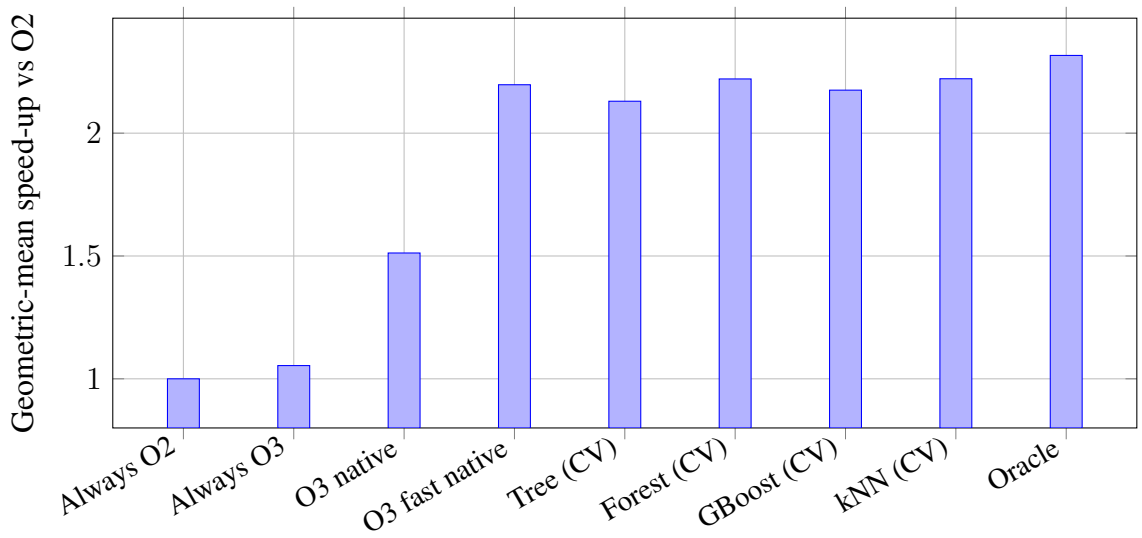


Figure 4: Geometric-mean speed-up over `O2` of the static configurations, the decision models (5-fold CV) and the oracle.

Under leave-one-family-out the speed-up of the model falls to 1.794 (69.6% of the oracle gain), the regret rises from 5.0% to 64.7%, and the model is *worse* than always choosing the aggressive configuration (2.197). Random splits overestimate the quality for new code because the corpus contains many close variants of the same kernel. This is the main finding about the generalisation of the approach: with this corpus the model is not yet reliable on a loop pattern that it has not seen.

4.7 Portable Setting

A more demanding decision arises when the program must run on any x86-64 machine and keep strict floating-point semantics. The decision space is then restricted to `O2`, `O3`, `O2_unroll4` and `O2_unroll8`, and the labels are recomputed from the same measurements with the same rule (Table **??**).

Table 9: Portable setting: only the four IEEE-safe, non-native configurations (`O2`, `O3`, `O2_unroll4`, `O2_unroll8`) are allowed.

Model	Protocol	Acc. (%)	GM speed-up	Oracle gain (%)	Regret (%)	Slow-downs
Always -O2	–	46.2	1.000	0	21.2	0
Always -O3	–	22.0	1.054	33	14.3	32
Decision Tree	5-fold CV	51.9	1.088	53	9.0	5
Decision Tree	LOFO	37.4	1.012	7	19.6	8
Random Forest	5-fold CV	52.1	1.102	60	7.3	7
Random Forest	LOFO	38.5	1.022	13	18.1	5
k-NN	5-fold CV	57.1	1.108	64	6.6	6
k-NN	LOFO	43.4	1.039	23	15.8	3
Oracle	–	–	**1.175**	100	0.0	0

Here the potential is smaller (oracle 1.175), but the decision matters: always using `O3` gives 1.054 with 32 kernels that become more than 5% slower than `O2`. The k-NN reaches 1.108 in cross-validation (63.5% of the oracle gain) with 6.4 slow-downs on average, i.e. it keeps most of the benefit of `O3` while avoiding most of its losses. Under leave-one-family-out it obtains 1.039 (23.5% of the oracle gain) with 3 slow-downs: this is slightly below the average speed-up of always using `O3`, but much safer.

4.8 Per-Family Analysis

Table 10: Per-family geometric-mean speed-up over `O2` of the selected model (k-NN).

Family	n	Oracle	Model (CV)	Model (LOFO)	Acc. CV (%)	Acc. LOFO (%)
`axpy`	36	1.88	1.82	1.76	67	53
`cond`	16	5.02	4.98	1.29	81	6
`dot`	18	4.19	4.19	3.74	94	89
`matmul`	24	2.83	2.76	1.46	75	4
`recur`	24	1.38	1.26	1.27	54	42
`stencil1d`	16	1.73	1.67	1.67	50	81
`stencil2d`	8	2.26	2.23	2.21	25	75
`strided`	20	1.24	1.07	1.16	25	30
`traverse2d`	20	3.71	3.65	3.67	80	80

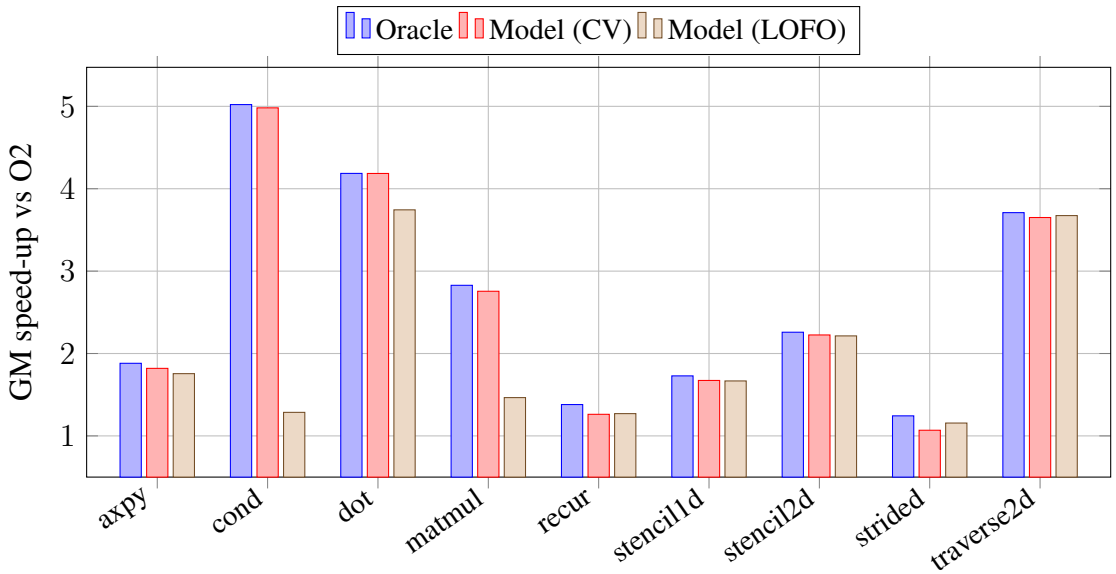


Figure 5: Per-family speed-up over `O2` for the k-NN: oracle, random-split cross validation, and leave-one-family-out.

Table **??** and Figure **??** show that families with a clear structural signature (`dot`, `traverse2d`, `matmul`, `cond`) are predicted nearly perfectly in cross-validation. The `strided` family is

the hardest: the best configuration depends on the interaction of stride, cache size and the hardware prefetcher, which the static features describe only roughly. In the leave-one-family-out test, `cond` and `matmul` lose most of their gain, because no similar loop was in the training set: this indicates that the corpus must contain more diverse loop patterns before the model can be trusted on new code. The families `stencil1d`, `stencil2d`, `traverse2d` and `dot` transfer well because their features resemble those of other families.

4.9 Confusion Matrix and Feature Importance

Table 11: Confusion matrix of the k-NN (5-fold cross validation).

True \ Pred	O2	O3	O2+unroll4	O2+unroll8	O3+native	O3+unroll+native	O3+fast+native
O2	10	0	0	1	8	4	2
O3	1	2	0	0	2	1	0
O2+unroll4	0	0	0	0	0	2	0
O2+unroll8	1	0	0	0	1	0	0
O3+native	2	1	0	1	45	13	3
O3+unroll+native	5	0	1	0	6	22	3
O3+fast+native	0	0	0	0	5	3	37

The confusion matrix (Table **??**) shows that most errors are between neighbouring aggressive configurations (for example `O3_native` versus `O3_unroll_native`), whose timing difference is small. Errors that choose a much slower configuration are rarer but produce the slow-downs counted in Table **??**.

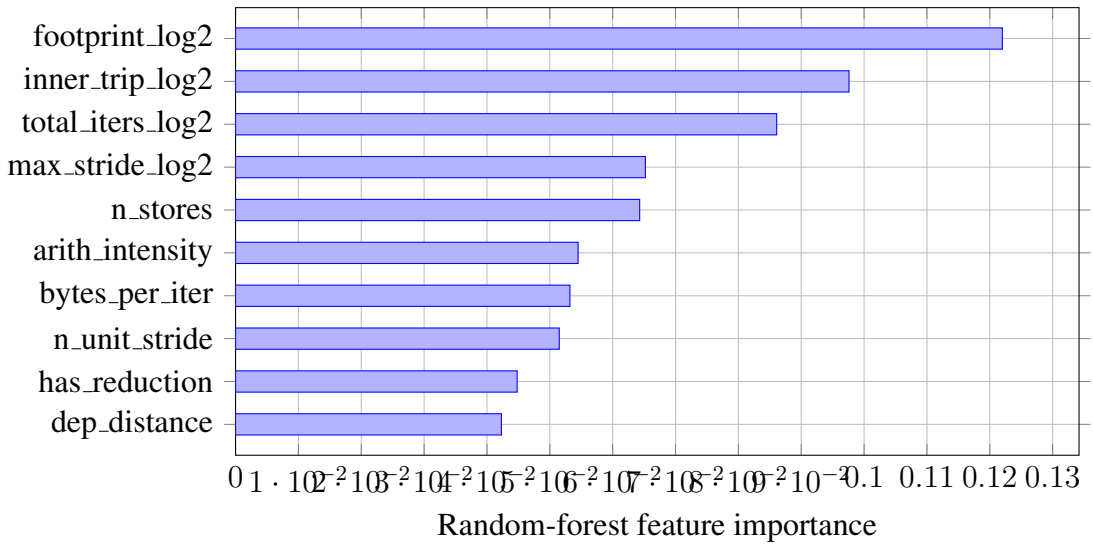


Figure 6: Ten most important loop features.

Figure **??** shows that working-set size, trip counts and the largest memory stride are the most important features. These are exactly the quantities that determine whether a loop is

bound by arithmetic, by cache or by memory bandwidth, and they are available from the front end without running the program.

4.10 Interpretable Rules

A depth-5 decision tree trained on all data gives readable rules; the first levels are listed below (`n_stores`=0 identifies reductions, which are sent to `O3_fast_native`).

```
|--- n_stores <= 0.50
|   |--- footprint_log2 <= 22.79
|   |   |--- class: O3_fast_native
|   |--- footprint_log2 >  22.79
|   |   |--- class: O3_fast_native
|--- n_stores >  0.50
|   |--- n_unit_stride <= 2.50
|   |   |--- tac_len <= 7.50
|   |   |   |--- footprint_log2 <= 21.58
|   |   |   |   |--- max_stride_log2 <= 6.79
|   |   |   |   |   |--- class: O3_unroll_native
|   |   |   |   |--- max_stride_log2 >  6.79
|   |   |   |   |   |--- class: O2
|   |   |   |--- footprint_log2 >  21.58
|   |   |   |   |--- bytes_per_iter <= 12.00
|   |   |   |   |   |--- class: O2
|   |   |   |   |--- bytes_per_iter >  12.00
|   |   |   |   |   |--- class: O2
|   |   |--- tac_len >  7.50
|   |   |   |--- dtype_bytes <= 6.00
|   |   |   |   |--- total_iters_log2 <= 14.50
|   |   |   |   |   |--- class: O3_fast_native
|   |   |   |   |--- total_iters_log2 >  14.50
|   |   |   |   |   |--- class: O3_fast_native
|   |   |   |--- dtype_bytes >  6.00
|   |   |   |   |--- n_loads <= 1.50
```

4.11 Overhead of the Model

Feature extraction takes about 0.7 ms and prediction about 1.3 ms per kernel, whereas a single GCC compilation takes about 71 ms on average. The decision is therefore of the same order as, or cheaper than, one compiler run and is negligible compared with the search of an iterative-compilation approach that compiles and runs every configuration (7 compilations per loop here).

4.12 Sample Compiler Program

The following C kernel (matrix product in the poor loop order j-k-i) is given to the advisor. Note that this kernel is part of the training corpus, so the example illustrates the tool and is not a test of generalisation.

```
1 static DT A[M][M], B[M][M], C[M][M];    /* M = 96, DT = double */
2
```

```
DT kernel(int r)
{
    for (int j = 0; j < M; j++) {
        for (int k = 0; k < M; k++) {
            for (int i = 0; i < M; i++) {
                C[i][j] += A[i][k]*B[k][j];
            }
        }
    }
    return C[0][0];
}
```

4.13 Lexical Analysis Output

Token stream (first tokens of the loop, KIND:lexeme) for the kernel `cond_double_8192_0p5_1` printed by the scanner:

```
KEYWORD:for     PUNCT:(         TYPE:int        IDENT:i
OP:=            NUMBER:0        PUNCT:;         IDENT:i
OP:<            IDENT:N         PUNCT:;         IDENT:i
OP:++           PUNCT:)         PUNCT:{         KEYWORD:if
PUNCT:(         IDENT:x         PUNCT:[         IDENT:i
PUNCT:]         OP:>            NUMBER:0.5      PUNCT:)
PUNCT:{         IDENT:y         PUNCT:[         IDENT:i
...
```

4.14 Three Address Code

The loop nest of the same branchy kernel (`if (x[i] > 0.5) y[i] = 0.5*x[i] + 0.1; else y[i] = 0.25*x[i];`) is lowered to the following three-address code (before any optimisation; note the repeated loads of `x[i]`, which common-subexpression elimination removes later):

```
i = 0
L1:
t1 = i >= N
if t1 goto L2
t2 = x[i]
t3 = t2 > 0.5
ifFalse t3 goto L3
t4 = x[i]
t5 = 0.5 * t4
```

```
t6 = t5 + 0.1
t7 = x[i]
t8 = t6 * t7
t9 = t8 + 0.2
y[i] = t9
goto L4
L3:
t10 = x[i]
t11 = 0.25 * t10
y[i] = t11
L4:
i = i + 1
goto L1
L2:
t12 = y[0]
return t12
```

4.15 Advisor Output

```
source file      : kernels/matmul_double_96_jki.c
tokens           : 97
loop nest        : depth 3, trip counts [96, 96, 96], vars ['j', 'k', 'i']
memory strides   : {'C[i][j]': 96, 'A[i][k]': 96, 'B[k][j]': 0}
feature vector   :
```

```
model            : k-NN
    P(O2_unroll4       ) = 1.00
advised config   : O2_unroll4
advised command  : gcc -O2 -funroll-loops --param max-unroll-times=4 -o kernel kernels/
    matmul_double_96_jki.c -lm

-O2 baseline     :     467163.6 ns / call
O2_unroll4       :     355592.7 ns / call   speed-up x1.31
```

The feature vector, printed between these lines, shows the strides (96 elements for `C` and `A`, i.e. a cache-hostile access), and the advisor selected `O2_unroll4`, which was measured to be faster than `-O2` for this kernel.

4.16 Limitations and Threats to Validity

- **Single machine:** the labels depend on this processor. The large gains of the `native` configurations come from its AVX2/AVX-512 units; a different CPU requires re-labelling and re-training.

- **Synthetic corpus:** the kernels are small, regular HPC loops rather than whole applications, and similar kernels appear in several sizes. Random cross-validation is therefore optimistic, as the LOFO results show.
- **Timing noise:** on a shared virtual machine the residual noise (Section above) is comparable to the 5% tie tolerance.
- **Numerical safety:** `-ffast-math` changes floating-point results. A configuration is only admissible if the checksum stays within a relative error of 10^{-3}, which is a coarse test and is not a proof of correctness for other inputs.
- **Language subset:** the front end handles a C subset (affine subscripts, constant bounds, one function), not full C.
- **Portability:** binaries built with `-march=native` run only on processors with the same instruction set.

5. Conclusion and Future Work

5.1 Conclusion

This report presented an ML-based loop-optimisation decision model built on a compiler front end that was implemented from scratch. The lexical analyser, the recursive-descent parser, the semantic and loop analyser and the three-address-code generator turn the source of a loop nest into 23 features (iteration space, memory strides, dependence distance, reductions, arithmetic intensity, working set). A supervised model uses these features to select one of 7 GCC optimisation configurations.

The data set was measured and not simulated: 182 kernels were compiled with every configuration and executed, and configurations that changed the numerical result were rejected. No static configuration was best for all loops. With random cross-validation, the best model (k-NN) reached a geometric-mean speed-up of 2.221 over `-O2`, 95.0% of the oracle's gain (2.316). The honest reference is the best static choice: always using `-O3 -march=native -ffast-math` gives 2.197 on this machine, so the model is only marginally better in cross-validation and worse (1.794) on loop families that were not in the training set. Where the binary has to be portable and IEEE-safe, the decision is more valuable: the model reached 1.108 against 1.054 for always using `-O3`, with far fewer slow-downs. Feature extraction and prediction cost less than one compiler invocation.

Overall, the study shows that the early compiler phases provide the information that optimisation decisions need, and that a learned decision can replace a fixed heuristic, with two caveats: the training corpus must cover the loop patterns that the model will meet, and on a machine where aggressive flags almost always win, a learned model adds little over simply using them.

5.2 Future Work

Future work will focus on (i) a larger and more diverse corpus, including real benchmark loops (for example from PolyBench), to close the gap between random and leave-one-family-out evaluation; (ii) a larger decision space with loop-level choices such as tile size, unroll factor and interchange, implemented as source-to-source transformations on the AST; (iii) repeating the experiment on several processors and adding machine features to the model; (iv) a more robust measurement setup on a dedicated machine with fixed frequency; (v) active learning, in which the model requests measurements only for loops it is unsure about; and (vi) integration with LLVM/MLIR passes.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson/Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed., Morgan Kaufmann, 2011.

[3] A. W. Appel, *Modern Compiler Implementation in ML*, Cambridge University Press, 1998.

[4] S. S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997.

[5] R. Allen and K. Kennedy, *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*, Morgan Kaufmann, 2002.

[6] D. F. Bacon, S. L. Graham, and O. J. Sharp, "Compiler transformations for high-performance computing," *ACM Computing Surveys*, vol. 26, no. 4, pp. 345–420, 1994.

[7] M. E. Wolf and M. S. Lam, "A data locality optimizing algorithm," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1991, pp. 30–44.

[8] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proc. ACM SIGPLAN PLDI*, 2008, pp. 101–113.

[9] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proceedings of the IEEE*, vol. 106, no. 11, pp. 1879–1901, 2018.

[10] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," *ACM Computing Surveys*, vol. 51, no. 5, article 96, 2018.

[11] M. Stephenson, S. Amarasinghe, M. Martin, and U.-M. O'Reilly, "Meta optimization: improving compiler heuristics with machine learning," in *Proc. ACM SIGPLAN PLDI*, 2003, pp. 77–90.

[12] M. Stephenson and S. Amarasinghe, "Predicting unroll factors using supervised classification," in *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2005, pp. 123–134.

[13] F. Agakov, E. Bonilla, J. Cavazos, B. Franke, G. Fursin, M. F. P. O'Boyle, J. Thomson, M. Toussaint, and C. K. I. Williams, "Using machine learning to focus iterative optimization," in *Proc. CGO*, 2006, pp. 295–305.

[14] G. Fursin et al., "Milepost GCC: Machine learning enabled self-tuning compiler," *International Journal of Parallel Programming*, vol. 39, no. 3, pp. 296–327, 2011.

[15] C. Cummins, P. Petoumenos, Z. Wang, and H. Leather, "End-to-end deep learning of optimization heuristics," in *Proc. International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2017, pp. 219–232.

[16] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: a machine learning guided compiler optimizations framework," arXiv:2101.04808, 2021.

[17] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.

[18] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[19] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 265–283.

[20] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[21] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[22] Free Software Foundation, *Using the GNU Compiler Collection (GCC)*, GCC Online Documentation.